

Helping Hands

Final project report - ELE 3921 Web Applications Development

Team members: [add names] | GitHub: [add public repository link] | Video: [add video link]

Project summary

Helping Hands is a Django web application designed to connect seniors who need help with everyday tasks to trusted local helpers who want to earn money by helping their community. The project addresses a practical local problem: many older adults can live independently, but still need occasional support with errands, technology, housework, transport, gardening, or small maintenance tasks.

The application has two main user roles: requesters and helpers. Requesters can create job posts, review applications, select a helper, message the selected person, mark jobs as completed, and leave reviews. Helpers can browse open jobs, apply with a short message, track pending applications, message requesters after selection, and build reputation through reviews.

Main features

- Registration and authentication using Django's built-in user system.
- Role-based accounts for requesters and helpers, with different dashboards and redirects.
- Job posting, category browsing, job details, applications, helper selection, and completion flow.
- Per-job messaging between the requester and selected helper.
- Profile pages with bio, location, avatar, posted/accepted jobs, and reviews.
- Review and report features after relevant job interactions.
- Admin interface for managing users, profiles, categories, jobs, applications, messages, reviews, and reports.
- Seeded demo data, uploaded avatars, Bootstrap, and a custom static stylesheet.

Architecture overview

Helping Hands relational schema

Main Django models and relationships used by the application

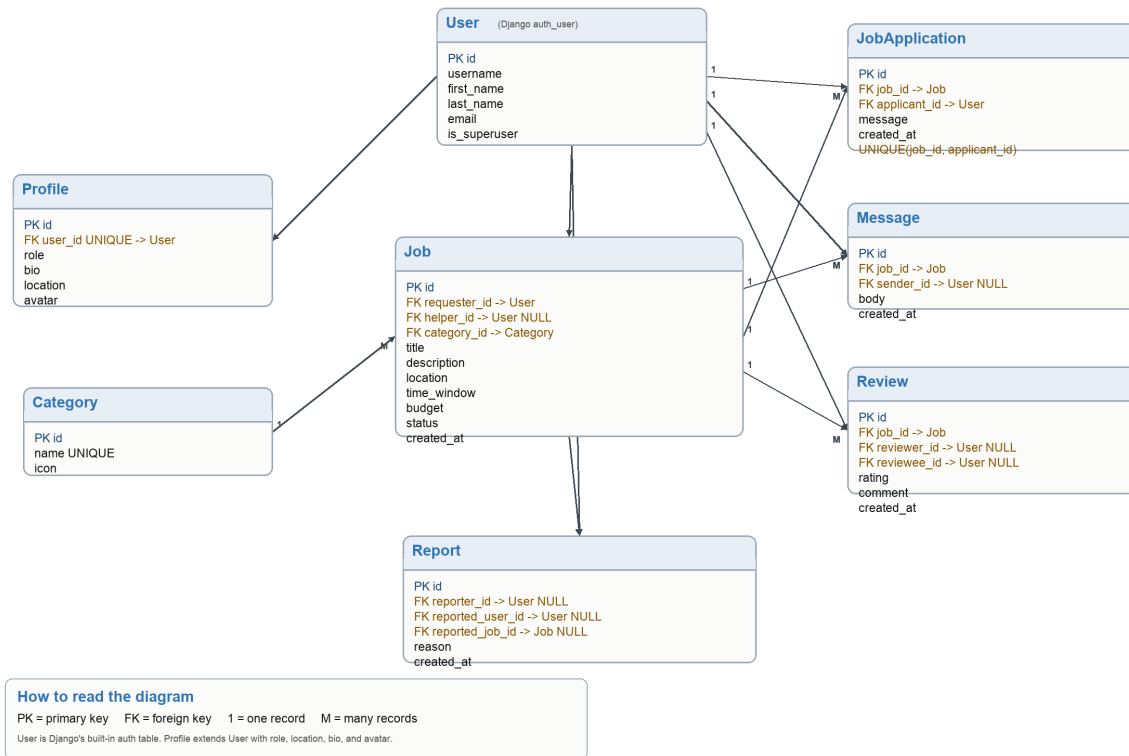


Figure 1. Relational schema for the Helping Hands application.

Models

Model	Purpose	Important relationships
Category	Stores job categories such as errands, cleaning, garden work, and tech help.	Referenced by Job with protected deletion.
Profile	Extends Django User with role, bio, location, and avatar.	One-to-one with User.
Job	Stores task posts, requester, helper, budget, timing, location, and status.	Requester/helper connect to User; category connects to Category.
JobApplication	Stores helper applications and optional application messages.	Connects Job and applicant User; unique per job/applicant pair.
Message	Stores messages in a job-specific conversation.	Connects sender User to a Job.
Review	Stores rating and comment after completed jobs.	Connects reviewer, reviewee, and Job.
Report	Stores reports for moderation and safety.	Connects reporter, reported user, and related job.

Views and templates

The project uses Django templates for server-side rendering. Most user actions are implemented as regular Django views with redirects and messages after successful changes. The main views include home, registration/login/logout, profile pages, dashboard pages, job list/detail/create/apply/select/complete, messaging, review creation, and report creation.

Forms and validation

The application uses Django forms and model forms for user input. RegisterForm extends UserCreationForm and creates both the User and Profile. JobForm, ProfileForm, MessageForm, ReviewForm, and ReportForm handle the main user interactions. Form validation errors are displayed in the templates, while successful actions use Django messages to give feedback.

Authentication and access control

Authentication is based on Django's built-in User model. Views that require a logged-in user use login_required. Additional permission checks are handled in the views: only a requester can select a helper or complete their own job; only the requester and selected helper can access a message thread; reviews can only be left for completed jobs; and duplicate reviews/applications are prevented.

Development process

The project was developed around the core marketplace workflow. The models were created first so the main relationships were clear, then views and templates were added around the requester and helper journeys. The design aims to feel calm and trustworthy for seniors while still being easy for helpers to scan and use.

Most functionality is server-side Django, with only light JavaScript for interface details such as the role picker during registration and the message box scroll behavior. This kept the application simple and aligned with the course focus on Django, templates, forms, authentication, and database relationships.

Front-end design

The interface uses Bootstrap together with a custom static CSS file. The visual style uses a dark navigation bar, off-white page background, white content cards, terracotta accent buttons, serif headings, and readable form styling. The home page uses a large image hero, while the app pages are more functional and compact.

A custom static stylesheet is included at helply/static/helply/css/style.css. This was added so the project has committed static assets instead of relying only on inline CSS and external CDNs.

Peer review reflection

The peer review highlighted several strengths: the model design, the requester/helper split, access control, the admin setup, messaging, and the report feature. It also pointed out areas to improve before submission, especially the need for a clear requirements file, data dump, and custom static assets.

After reviewing the feedback, the project was cleaned up for submission. The repository now includes requirements.txt, a cleaner data dump, custom static CSS, and a README with setup instructions and sample credentials. JobApplication was also added to the admin interface so all important project data can be managed through Django admin.

Future improvements

- Add search and filtering by location, budget, time window, and review rating.

- Add notifications by email or in-app alerts when someone applies, is selected, or sends a message.
- Add stronger trust features such as identity verification, references, and admin moderation tools.
- Add payment handling or clear payment confirmation for real marketplace use.
- Improve mobile responsiveness further and add more automated tests for critical permission flows.

Sample user credentials

The following demo accounts can be used to test the application. All use the password **password123**.

Role	Username	Purpose
Admin	louie	Superuser account for Django admin.
Requester	bjorn_h	Senior/requester account for posting and managing jobs.
Requester	ragnhild_s	Requester account with example job activity.
Helper	magnus_a	Helper account for browsing and applying to jobs.
Helper	soupy	Additional helper account for testing role-specific flows.

Submission links and files

- GitHub repository: [add public repository link]
- Video presentation: [add video link]
- Data dump: project/data.json and project/fixtures/data.json
- Requirements file: project/requirements.txt
- Static CSS: project/helply/static/helply/css/style.css

AI tool usage

AI assistance was used for review and cleanup support near the end of the project: comparing the assignment requirements with the project, identifying missing submission checklist items, moving existing CSS into a static file, registering a missing admin model, preparing cleaner submission notes, and drafting this report. The core project concept, Django architecture, models, views, and workflows should be understood and explained by the team during the presentation.